《世界迷雾》逆向工程系列——数据

Original 维icc 迷雾资料库 Jan 28, 2022, 4:07 AM

这是本系列里**最有价值**的一篇。理解之后迷雾数据对你来说就不是秘密了，可以基于存档进行二次开发，生成各种酷炫的轨迹，也不怕鸽礼士跑路了。

然后这篇文章适合有**一点点**计算机基础的朋友，其他朋友就当作看热闹了。

长话短说，和上一篇文章一样，拿具体的存档作为样例，带你从**f3c2lotjwkxk**这个小文件到下面这张**8192x8192**的迷雾轨迹无码裸照。



同样，大部分时候只放结论，不放过程，因为解释不清楚。

以**16进制**打开这个文件后，看到的是这样的数据（这是我最大的文件，一共452KB数据）

```
785e ec5d 079c d444 db9f f4ba 9b6c caee
52ef e0e8 281c bdc3 d17b af2a e5e8 48ef
2020 2c1d 942a 2288 0828 2a28 2a20 1614
9405 1450 698a 080a c221 08a8 4811 94ce
7db3 93c9 b29b dbe5 0e2c 9faf f2fc 2ef9
3f7d 6692 c964 522e 0b00 0024 a000 0d6e
1303 58c0 011e 0840 0477 2209 c8c0 05dc
4001 6a06 9b07 6811 920e 0c60 022f f021
c90f b281 ec20 4784 3d27 5ae7 02b9 4102
4804 7940 5e28 2581 7c20 3f28 000a 42be
1028 0c8a 80a2 e001 c83f 0862 5331 50dc
a149 8e92 4a80 92a0 1428 edf0 2903 ca82
72a0 3ca8 002a 824a c049 9541 1550 1554
```

世界迷雾APP

众所周知，785E 是 zlib压缩方式的开头标识，如下图。那么我们找一个zlib库来解压一下，基本每个语言都有这样一个库，我们来试试python的zlib库。

Level	ZLIB	GZIP
1	78 01	1F 8B
2	78 5E	1F 8B
3	78 5E	1F 8B
4	78 5E	1F 8B
5	78 5E	1F 8B
6	78 9C	1F 8B
7	78 DA	1F 8B
8	78 DA	1F 8B
9	78 DA	1F 8B

世界迷雾APP

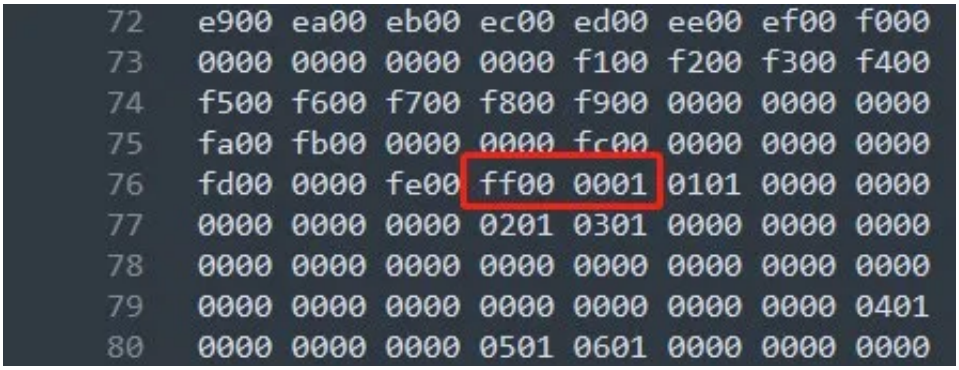
解压成功，可以99%确定就是这个格式。同时得到下面完整的数据（3.1M的数据，压缩率85%左右，特别高，说明空白特别多）。

```
0000 0000 0200 0300 0400 0000 0000 0000
0000 0000 0500 0600 0700 0800 0900 0a00
0000 0000 0000 0000 0000 0000 0000 0000
0000 0000 0000 0000 0000 0000 0b00 0c00
0d00 0e00 0f00 1000 0000 0000 0000 0000
0000 0000 0000 0000 1100 1200 0000 0000
0000 0000 0000 1300 1400 1500 1600 1700
0000 0000 1800 1900 1a00 1b00 0000 0000
0000 0000 0000 1c00 0000 0000 1d00 1e00
1f00 2000 2100 2200 0000 2300 2400 2500
2600 2700 0000 2800 2900 2a00 2b00 2c00
```

一下子就**豁然开朗**有没有！可以依稀看到0100，0200，0300，0400...

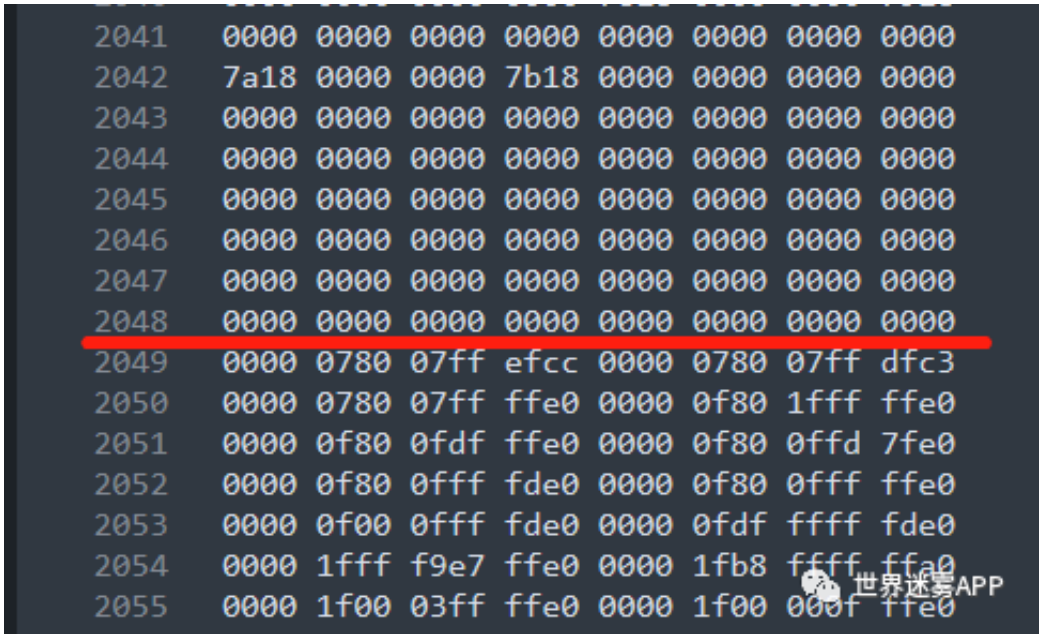
顺着规律往下找，后面是0800，0900，0a00，0b00...，这个简单，16进制嘛。

再继续往下，直到ff00，后面是0001，哟，进位放到了后面去了



继续往后翻页，直到出现理解不了的东西，同时注意了，**2048**行！

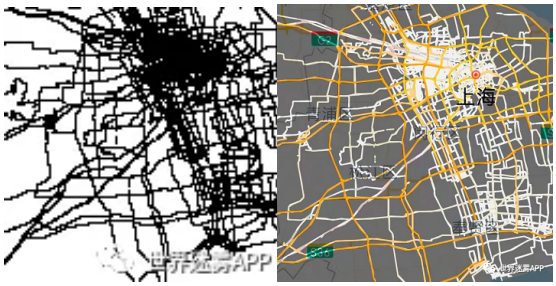
同时最后一个数字是7b18, 也就是$1x16^3 + 8x16^2+7x16+11$ = **6267** （后续用）。



2048和1024一样是个程序里常见的数字，又，我们知道这里每个数字占用4个字符，我这里一行是32个字符，8个数字，那么一共就是 2048 x 8 = **16384**个数字。

正好是128 x 128

我们把这个128 x 128 个数字做个简单模拟，空白的地方(0000)留空，有数字的地方画个点，到128就换一行，映射到一张128 x 128的图片上试试看。



Bingo，完全一样，离真相只有**一步**之遥了。

后面就是这‘一步’了

我们继续回到16进制文件，拉到最后：

```
203758   0000 8000 0000 0000 0000 8000 0000 0000
203759   0000 8000 0000 0000 0000 8000 0000 0000
203760   0000 8000 0000 0000 0000 8000 0000 0000
203761   0000 8000 0000 0000 0000 8000 0000 0000
203762   0000 8000 0000 0000 0000 8000 0000 0000
203763   0000 8000 0000 0000 0000 8000 0000 0000
203764   0000 8000 0000 0000 0000 8000 0000 0000
203765   0000 8000 0000 0000 0000 8000 0000 0000
203766   0000 8000 0000 0000 0000 8000 0000 0000
203767   0000 8000 0000 0000 0000 8000 0000 23c0
203768   81
```

世界迷雾APP

一共是**203768**行零**2**个字符，一共是：

203767 x 32 + 2 = 6,520,546 个字符

除去前面：2048 x32 = 65,536个字符

还有：6,520,546 – 65,536 = 6,455,010 个字符

插一句，这里2个字符=1个字节(byte)=8个比特(bit)，写程序的同学注意了。

那么，**顺其自然**的，我们猜想后面的数字要分给前面这**6267**个点上去，也就是每个点是

6,455,010 / 6267 = 1030

注意，后面没有小数点！

说明就是这样的！！

继续的，**我们知道**，这就是个映射表，比如0100就是第一块1030数据，ff00就是第255块1030数据，空白的地方继续留白。

取第一块数据，隐隐约约的，我们好像看到了什么，模模糊糊的一个图像！

```
2048  0000 0000 0000 0000 0000 0000 0000 0000
2049  0000 0780 07ff efcc 0000 0780 07ff dfc3
2050  0000 0780 07ff ffe0 0000 0f80 1fff ffe0
2051  0000 0f80 0fdf ffe0 0000 0f80 0ffd 7fe0
2052  0000 0f80 0fff fde0 0000 0f80 0fff ffe0
2053  0000 0f00 0fff fde0 0000 0fdf ffff fde0
2054  0000 1fff f9e7 ffe0 0000 1fb8 ffff ffa0
2055  0000 1f00 03ff ffe0 0000 1f00 000f ffe0
2056  0000 1f00 0001 ffe0 0000 1e00 0003 ffe0
2057  0000 3e00 0000 67d0 0000 3e00 0000 36d0
2058  0000 3e00 0000 1ad0 0020 3e00 0000 06d0
2059  0030 3e00 0000 06d0 0030 3bc0 0000 06d0
2060  003c 3b3f fc00 02d0 e03c 7a07 8000 02d0
2061  fffc 7f88 6000 03f0 ffff ff7c 2000 03f0
2062  ffff fffb e000 03f0 ffff ffff ff00 03f0
2063  b7ff ffff ffe0 0ff0 3fff ffff ffff ffd0
2064  0bff ffff fff7 e3e0 87ff fb5f ffff ffc0
2065  c7ff fa73 bfff ffc0 7fff fd9f f9ff ffc0
2066  01ff fc9c 977e 1f00 03ff fc90 981b 0000
2067  01ff fc91 8fbf 0000 47ff fc97 c0e1 e000
2068  74ff fc9f 60fc 1800 dd7b f8bc 3f83 0600
2069  005f fdb8 0001 e600 001f fd38 0000 3c00
2070  001f fd3f 8000 0000 001f ff68 8000 0000
2071  002f feb9 8000 0000 002f fbb1 8000 0000
2072  002f f730 8000 0000 002f f760 0000 0000
2073  002f f760 0000 0000 007f f760 0000 0000
2074  007f f760 0000 0000 007f ee60 0000 0000
2075  007f e6a0 0000 0000 005f e6a0 0000 0000
2076  005f efa0 0000 0000 00ef efa0 0000 0000
2077  00bf eea0 0000 0000 00bf fca0 0000 0000
2078  00ff ed20 0000 0000 017f cd60 0000 0000
2079  01ff cd40 0000 0000 01ff cde0 0000 0000
2080  03ff fda0 0000 0000 02ff dfc0 0000 0000
2081  23cc 6701 c3e0 3e00 0003 0000 7cd0 0dc0
```

世界迷雾APP

结合1030 约等于 **32x32 + 6**，又一个字符其实是4个比特（bit），一共是32x32x4个bit数据，那么**显而易见**，这可能就是64x64的一张点阵图，我们用黑点来表示比特1，空白表示比特0，生成一个64x64的点阵图：



Bingo!

再取第二、第三块同样处理，往旁边的轨迹拼一拼：得到了更长的轨迹！



稍微解释一下，这里需要把16进制换成2进制来处理：

16进制 –> 2进制
0x1 = 0b0001
0x2 = 0b0010
...
0xf = 0b1111

比如0x027f,
就是 0b 0000 00**1**0 0**111 1111**
也就是“6个空白**（1个点）**空白**（8个点）**”
点阵图就是： * ******

再把6267个64x64图片按照对应的位置拼起来，就成了这张8192 x 8192的**迷雾性感无码裸照**：



对了，群里大佬说，剩下那6个字符（3个字节）是表示国家代码，我还没有详细研究，不影响处理数据，这里我是直接跳过去了。

最后，附赠用于生成裸照的python代码一份：

使用前需要安装pillow库

把存档放在同一个目录，然后修改第4行

```
1 import zlib
2
```

```
# 使用前先安装pillow库
from PIL import Image

file_name = "f3c2lotjwkxk" # 存到放在同一个目录，这里改成你要处理的存档文件

# 1 byte = 8 bit
# data
# 128 * 128 * 2 byte: 2 byte => Map N
# 2 byte 12 : 0x21 N = 33
MAP_COL = 128
MAP_CNT = 128*128
START_BYTES = 128*128*2

# Map N
# 2048 * 8 * 2 byte
# 1030 * (N-1) ~ 1024 byte
# 3 byte 校验、面积
MAP_BIT = 64
MAP_BYTES = int(64*64/8)
SUM_CHECK = 3

with open(file_name, 'rb') as f:
    data = zlib.decompress(f.read())
    # print("data lenth:", str(len(data)))

    # with open(file_name + '.bin', 'wb') as f2:
    #     f2.write(data)

def get_map_byte_start(n):
    return START_BYTES + (MAP_BYTES+SUM_CHECK)*(n-1)

def get_map_index(x, y):
    idx = x+y*MAP_COL
    return data[idx*2+1]*256 + data[idx*2]

```

```
38 WHITE = 255
39 BLACK = 0
40
41 def print_overview():
42     image_data = []
43
44     image_data = [BLACK if i else WHITE for i in data[0:START_BYTES:
45
46     img = Image.new("1", (MAP_COL, MAP_COL), 0)
47     img.putdata(image_data)
48     img.save(file_name+".ov.png")
49
50 def get_image_data(x, y):
51     image_data = []
52
53     map_n = get_map_index(x, y)
54     if map_n == 0: return []
55
56     byte_start = get_map_byte_start(map_n)
57     # print(byte_start)
58
59     temp_bin = ""
60     for i in range(byte_start, byte_start+MAP_BYTES):
61         s = data[i]
62         temp_bin += "{:0>8b}".format(s)
63
64     return [BLACK if i=='1' else WHITE for i in temp_bin]
65
66
67 def print_area(x0=0, x1=MAP_COL, y0=0, y1=MAP_COL):
68     x_offset = 0
69     y_offset = 0
70
71     total_img = Image.new('L', (MAP_BIT*(x1-x0), (MAP_BIT*(y1-y0))),
72
```

```
    for y in range(y0, y1):
        y_offset = MAP_BIT * y
        for x in range(x0, x1):
            x_offset = MAP_BIT * x
            image_data = get_image_data(x, y)
            if not image_data: continue
            img = get_image(image_data)
            total_img.paste(img, (x_offset, y_offset))

    total_img.show()
    total_img.save(file_name+".png")

def get_image(image_data):
    img = Image.new("1", (MAP_BIT, MAP_BIT), 0)
    img.putdata(image_data)
    return img

if __name__ == '__main__':
    print_overview()
    print_area()
```

没有注释，靠自己领悟，出现任何技术问题，请不要求助我

Like the Author

“ 支持我，就请我喝杯咖啡☕ 吧 ”

迷雾资料库 · Table of Contents

Previous
《世界迷雾》逆向工程系列——文件

Next
迷雾橡皮擦 v2.0